Data Engineer | Interview Preparation

Role-level guide from aggregated job descriptions

- Focus: Python, SQL, ETL/ELT, AWS pipelines and data quality
- Architecture / troubleshooting / coding / communication
- Practice with real examples; do not claim tools not used in production

Target Role & Match

Supported experience and honest gaps

- Strong: Python, SQL, AWS Glue, S3, Lambda, Step Functions, REST APIs
- Strong: incremental processing, data validation, ETL, CloudWatch monitoring
- Gaps: Snowflake, dbt, Databricks, Azure ADF/Fabric, GCP
- Gaps: Kafka, advanced Spark optimization, Airflow in production
- Transfer: discuss equivalent AWS implementations, not invented experience

60-Second Introduction

Use as a template; adapt to the opening

- I am a Python and Data Engineer with experience delivering finance-data ETL
- I have used AWS Glue and PySpark with S3, Lambda and Step Functions
- I build validation, API integrations, alerts and reproducible workflows
- Previously I delivered Flask/MySQL business applications and SQL tuning
- I am interested in reliable, scalable pipelines and production ownership

2-Minute Introduction

Expand the story with concrete responsibilities

- Background: full-stack Python/MySQL to contract data engineering
- Finance pipeline: source extraction, mappings, incremental loads, outputs
- Document automation: event-driven queue processing, ORDS, templates
- Quality: validation checks, monitoring, exception handling, replay
- Conclude: what is demonstrated and what you are actively learning

Project Deep Dive 1

Finance pipeline | explain the data flow

- Source -> S3 landing -> Glue/PySpark transform -> downstream datasets
- Explain bookmarks, transformation logic, schema checks and retries
- Describe Step Functions orchestration, Lambda triggers and alerts
- Discuss what 30GB+/day means; separate factual and estimated metrics
- Prepare to explain security, PII, cost, partitioning and idempotency

Project Deep Dive 2

Serverless document workflow | explain trade-offs

- Trigger or queue -> Lambda -> ORDS/DynamoDB enrichment -> PDF/CSV
- SQS retries and dead-letter queue: know design, do not overclaim
- Idempotency keys, reprint handling, deterministic file naming
- Distinguish storage of artifacts from database state
- Discuss 80% manual-effort reduction and measurement assumptions

Project Deep Dive 3

Business applications | SQL & APIs

- Flask REST endpoint design: contracts, validation, auth and errors
- MySQL views, stored procedures, indexes and query plans
- Explain operational reporting and how source tables relate
- Discuss pagination, transactions and appropriate constraints
- Describe how to debug slow SQL and API timeouts

SQL & Data Modeling

High-frequency technical questions

- INNER vs LEFT JOIN; many-to-many duplication risks
- Window functions: ROW_NUMBER, DENSE_RANK, running totals
- Slowly changing dimensions: Type 1 vs Type 2
- Incremental loads: watermarks, upserts, deduplication
- Data reconciliation: row counts, control totals, null checks

Python & PySpark

Practice implementing without assistance

- Read CSV/JSON; normalize schema; handle nulls and bad records
- GroupBy aggregation vs window; joins and skew
- Spark lazy execution, shuffles, partitions and caching
- Write unit tests for transformations and invalid inputs
- Explain when Pandas fits vs distributed processing

Production Troubleshooting

Answer with detection, containment and root cause

- Upstream API late: backoff, retries, notifications, SLA
- Duplicate records: idempotent keys, merge/upsert, audit trail
- Changed schema: quarantine invalid records, version mappings
- ETL latency spike: identify stage, shuffle, source and sink
- Recover: targeted rerun, reconciliation, documented incident

System Design Exercise

Design an invoice ETL with daily and event streams

- Ingress: API/file/SQS into S3 raw zone
- Processing: Glue/PySpark, metadata and validation checks
- Storage: curated S3 partitions; suitable serving database
- Orchestration: Step Functions, alerts and replayability
- Consider secrets, IAM least privilege, encryption and costs

Coding Frameworks

Explain approach, then implement and test

- SQL: latest invoice per customer with ROW_NUMBER()
- SQL: reconcile order totals against settlement records
- Python: validate records and generate exception output
- PySpark: deduplicate keys by event timestamp
- Check edge cases: nulls, empty input, duplicates and timezone

Behavioral STAR

Structure each answer around a verified event

- Situation: legacy workflow, deadline, data consistency issue
- Task: own a defined integration or processing component
- Action: design, build, validate, monitor and collaborate
- Result: supported impact; explain how it was measured
- Never fabricate a major outage, leadership scope or metrics

Final Revision Checklist

Prioritize gaps and credible learning plans

- Rehearse two data pipelines from source to destination
- Solve 5 SQL queries and 2 Python transformation exercises
- Review Glue bookmarks, Spark shuffles and AWS IAM
- Prepare one small Snowflake/dbt or Databricks POC honestly
- Tailor discussion to each actual job and location constraint